

AI Email Inbox Assistant — BRD & High-Level Design

Document	Business Requirements Document (BRD) + High-Level Design (HLD)
Version	0.2 — updated to repository commit f16119f ("Strengthen persistence privacy and test coverage")
Date	19 September 2026
Team / Author	Group 14 — Krishna Tandon, Mridul Arora, Mugdha Deshpande, Pranati Sankalkar, Pratham Upadhyay
Status	Prototype implemented; production items marked <i>Planned</i> . Statuses re-verified against the code and a passing test run (5 tests)

1. Executive summary

Knowledge workers lose a large part of their day deciding *which* emails matter, *what* each one asks of them, and *how* to answer without over-committing. The **AI Email Inbox Assistant** turns an inbox into a prioritised, explained, actionable workspace:

1. **Triage** — every email gets a summary, a priority (Urgent / High / Normal / Low) with a stated reason, sender sentiment, and extracted tasks with deadlines.
2. **Respond** — a draft reply in a suitable tone, with a **confidence rating and a human-review flag** so the tool knows when *not* to be trusted (pricing, commitments, upset senders).
3. **Ask** — natural-language questions across the inbox ("What are my upcoming deadlines?"), answered *only* from email content, with source emails cited.
4. **Act** — deadlines are resolved from natural language ("by Friday", "tonight") into real datetimes and exported as a standard [.ics](#) calendar file.
5. **Monitor** — an **Inbox Health Score** summarises workload stress at a glance.

Design stance: the assistant *recommends and drafts*; the human *decides and sends*. There is no auto-send and no mailbox write access. Nothing is sent to the model until the user gives consent, and email addresses and phone numbers are redacted first.

Part A — Business Requirements (BRD)

2. Business problem

Pain point	Consequence
High email volume with no reliable sense of what is urgent	Critical items (outage, angry client) wait behind newsletters
Tasks and deadlines are buried in prose	Missed or late commitments
Writing replies is slow, and hasty replies over-commit (prices, dates)	Rework, reputational and contractual risk
Answering "what did X say about Y?" means manual searching	Time lost; context missed
Deadlines in email never reach the calendar	Manual copy-paste, or forgotten

3. Goals and success measures

#	Business goal	Success measure
G1	Cut time spent triaging email	Median minutes from "open inbox" to "know what needs action today"
G2	Reduce missed deadlines	Deadlines extracted vs. deadlines actually in the email (golden set)
G3	Speed up replies safely	Share of drafts accepted with $\leq$ minor edits, and zero risky drafts flagged "safe"
G4	Trustworthy answers	Every "Ask My Inbox" answer cites real emails;

#	Business goal	Success measure
		unanswerable questions are refused
G5	Reduce calendar friction	Deadlines converted to calendar entries in one click

4. Stakeholders and personas

Persona	Needs
Busy manager / team lead (primary)	Fast triage, safe reply drafting, deadline awareness
Account / customer-facing manager	Spot frustrated clients early; avoid risky commitments on price/dates
Executive assistant (secondary)	Ask questions across a principal's inbox; export dates to calendar
IT / Security / Compliance	Assurance on PII handling, no uncontrolled auto-actions, auditability

5. Scope

In scope (implemented prototype) — single user; 8 synthetic emails from a JSON file, seeded into a local SQLite database; analysis, drafting, inbox Q&A, health score, calendar `.ics` export; results persisted locally; consent gate and contact-identifier redaction; web UI.

Out of scope (this release) — sending email; live mailbox connection (Gmail / Microsoft 365); attachments (e.g. the pricing proposal "attached" in email e2 is not read); multi-user accounts; mobile app.

Planned — see the roadmap in §21.

6. Functional requirements

ID	Requirement	Priority
FR-01	Load and display an inbox; select an email to read	Must

ID	Requirement	Priority
FR-02	Analyse an email → summary, priority + reason, tasks with optional deadline, sender sentiment, suggested reply tone	Must
FR-03	Group inbox by priority; show "Unanalyzed" until analysed	Should
FR-04	"Analyze all emails" in one action	Should
FR-05	Inbox Health Score from urgency and sender-stress counts	Could
FR-06	Draft a reply using the analysis; provide confidence (High/Medium/Low), reason, and human-review flag; draft is editable	Must
FR-07	Answer natural-language questions using only inbox content; cite sources; refuse when not answerable	Must
FR-08	Resolve deadlines to real datetimes anchored to the email's timestamp; never invent dates; export as <code>.ics</code>	Should
FR-09	Use earlier emails in the same thread as context	Should
FR-10	Deadline timeline view across all analysed emails	Could
FR-11	Persist analyses, drafts, and calendar plans between sessions	Should

ID	Requirement	Priority
FR-12	Connect a live mailbox (OAuth, read-only first)	Should
FR-13	Multi-language emails and replies	Could
FR-14	Audit log of AI outputs and user decisions	Should
FR-15	Never send email automatically	Must
FR-16	Require user consent before any AI request	Must
FR-17	Redact direct contact identifiers before data leaves the app	Must

7. Non-functional requirements

Category	Requirement	Target
Performance	Single-email analysis or draft returns within a usable time	$p95 \leq 5 \text{ s}$ (<i>measure</i>)
Performance	Repeat views do not re-call the LLM	Cache hit = 0 API calls
Reliability	LLM/API failure never crashes the UI	Graceful message + retry
Accuracy	Priority agrees with human labels on a golden set	$\geq 85\%$
Groundedness	Inbox answers cite only real emails	100% valid IDs
Security	No secrets in source control	0
Security	All model/user text escaped before rendering	100%

Category	Requirement	Target
Privacy	PII minimised before leaving the trust boundary; consent obtained	Redaction step + consent
Usability	Explanations shown with every AI output (why this priority, why this confidence)	Always
Maintainability	Separated concerns; typed contracts	—
Testability	Automated tests for deterministic logic; eval set for LLM logic	≥ 70% on pure functions
Scalability	Handle real mailbox sizes	See §18

8. User stories

Expected behaviour on the demo dataset. *Verify each with the test plan in §19.*

ID	Story
US-01	As a manager, I want an outage escalation flagged immediately (e1).
US-02	As an account manager, I want an angry client surfaced (e4).
US-03	As a manager, I don't want a pricing reply to slip out unchecked (e2).
US-04	As a user, I want noise ignored (e5).
US-05	As a user, I want relative dates resolved correctly (e6).
US-06	As a user, I want answers I can trust.
US-07	As a user, I want thread context (e1 → e8).

9. Assumptions, constraints, dependencies

Assumptions *(validate with users)*

- A1. Users are willing to review AI drafts and will not want auto-send in v1.
- A2. Emails are mostly English in the pilot cohort.
- A3. Users accept a cloud LLM processing email text, subject to the privacy controls in §17.

Constraints

- C1. Hackathon time-box; synthetic data only.
- C2. Model calls go to Google's Gemini API; model is `gemini-3.5-flash-lite` (a stable low-cost model with structured-output support).
- C3. Calendar integration is file-based (`.ics`) — no OAuth, no calendar write scope.

Dependencies — Gemini API availability and quota (free-tier rate limits apply); Streamlit; `google-genai`, `pydantic`, `python-dotenv`, `pytest`; SQLite (Python standard library).

10. Risk register

#	Risk	Likelihood	Impact	Mitigation
R1	LLM hallucinates a fact, date, or commitment	Medium	High	"Do not invent" prompt rules; confidence + human-review flag; no auto-send; add validation and eval set
R2	Prompt injection via a malicious email	Medium	High	Delimit untrusted content; system instruction; validate outputs; no tool actions available to the model
R3	PII sent to a third-party model	High	High	<i>Implemented in part</i> : consent gate,

#	Risk	Likelihood	Impact	Mitigation
				email/phone redaction. <i>Still needed:</i> wider redaction, paid tier or enterprise terms with no-training guarantee, retention policy
R4	API key exposure	Occurred (earlier version)	High	Removed from repo and not in git history; rotate the key that was in the shared archive; add secret scanning (§17 T1)
R5	API outage / rate limiting	High (free tier)	Medium	Cached and persisted results (done); friendly errors (partial); retry with backoff and batch API (todo)
R6	Wrong priority erodes trust	Medium	Medium	Show the reason; let user override; measure on golden set
R7	Cost grows with inbox size (whole inbox sent per question)	High at scale	Medium	Retrieval (top-k) instead of full context; context caching

11. Business value and ROI

Value drivers: time saved on triage and drafting; fewer missed deadlines; earlier detection of at-risk clients; consistent, careful replies.

ROI model — replace the illustrative inputs with pilot measurements:

Annual value = Users × Emails/day × Minutes saved/email × Working days × Hourly cost ÷ 60

Input	Illustrative value (<i>assumption — replace</i>)
Users	100
Emails handled per user per day	60
Minutes saved per email	0.4 (e.g. 40% of ~1 min triage + reply thinking)
Working days per year	220
Loaded hourly cost	\$30
Annual value	100 × 60 × 0.4 × 220 × \$30 ÷ 60 = \$264,000

Subtract: LLM usage cost (emails × calls per email × tokens × price — take price from the current Gemini pricing page), hosting, and integration/maintenance effort. Each email costs at most 3 model calls (analyse, draft, calendar) plus one per inbox question.

Adoption path: read-only mailbox connector → pilot with one team → measure G1–G5 → add write-back (drafts saved to the mail client's Drafts folder) only after trust is established.

Competitive context to address in the pitch: built-in assistants in Gmail and Outlook, and dedicated email clients. Differentiators to emphasise: *explicit confidence + human-review gating*, *grounded and cited inbox Q&A*, and *timestamp-anchored deadline extraction that refuses to guess*.

12. Traceability matrix

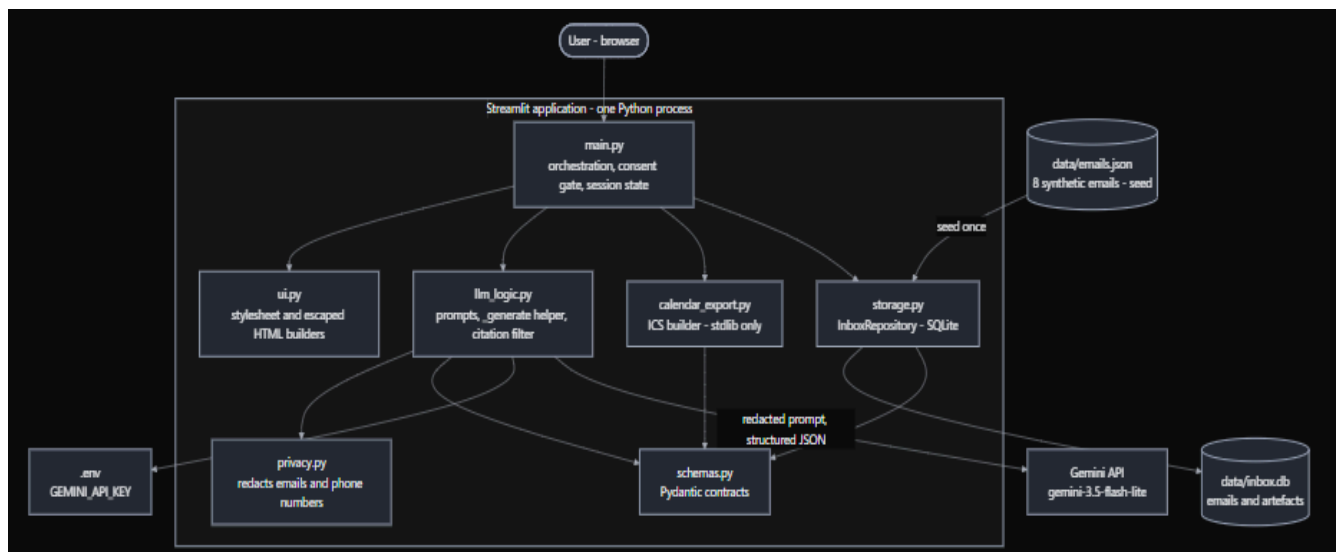
Requirement	Design element (HLD)	Test (§19)
FR-02, FR-03	<code>analyze_email</code> + <code>EmailAnalysis</code> schema	T-LLM-01, T-UI-01
FR-05	Health score formula	T-UNIT-01

Requirement	Design element (HLD)	Test (§19)
FR-06, FR-15	<code>draft_reply</code> + <code>DraftReply</code> ; no send path	T-LLM-02, T-SEC-04
FR-07, G4	<code>answer_inbox_question</code> + source validation	T-LLM-03, T-LLM-04
FR-08	<code>plan_calendar_events</code> + <code>calendar_export</code>	T-UNIT-02...04, T-LLM-05
FR-09	Thread stitching (planned)	T-LLM-06
FR-11	<code>storage.py</code> <code>InboxRepository</code>	T-STORE-01
FR-16, FR-17	Consent gate in <code>main.py</code> ; <code>privacy.py</code>	T-PRIV-01...03
NFR Security	Threat model §17	T-SEC-01...04

Part B — High-Level Design (HLD)

13. System architecture

13.1 Current architecture (implemented)

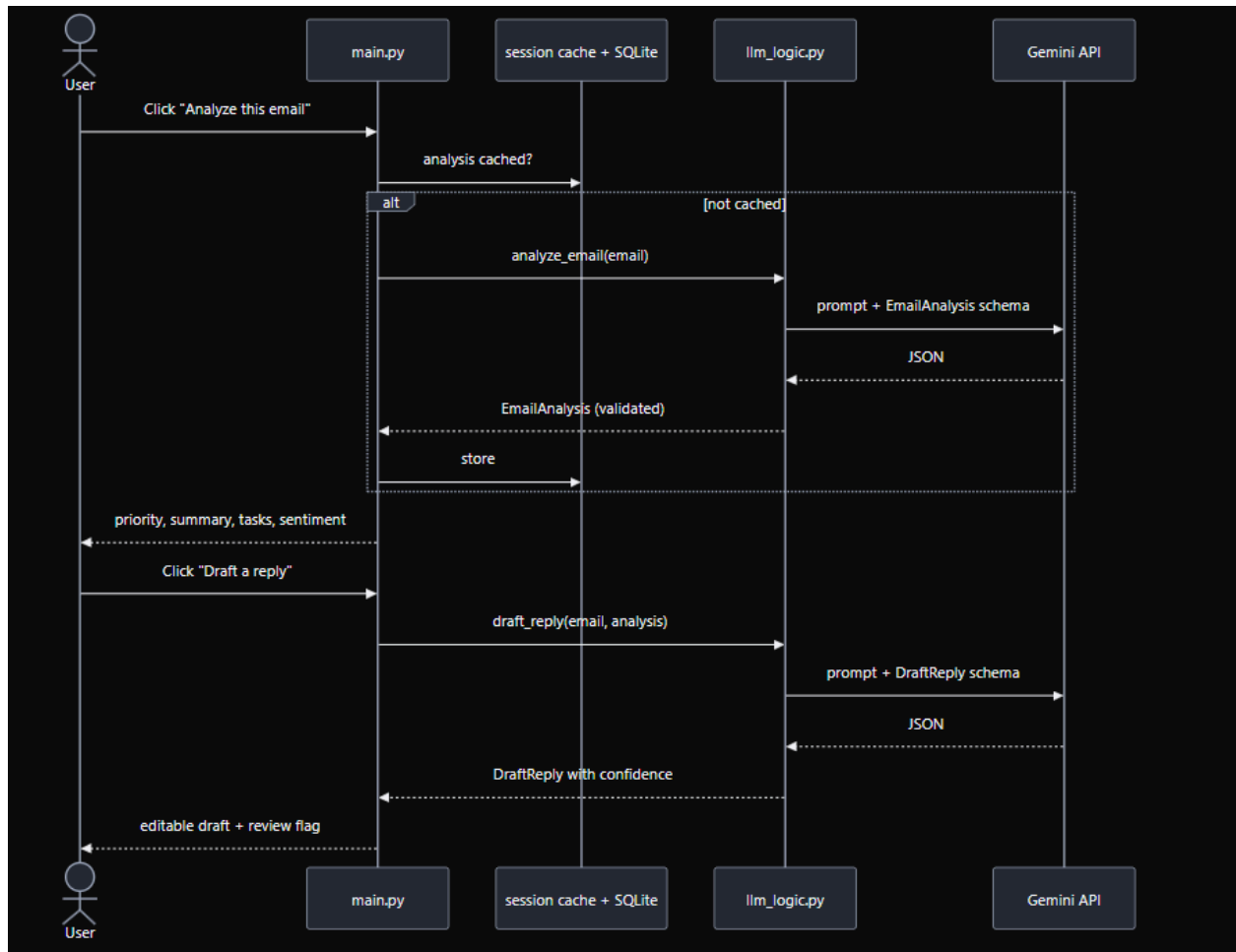


Key design decisions

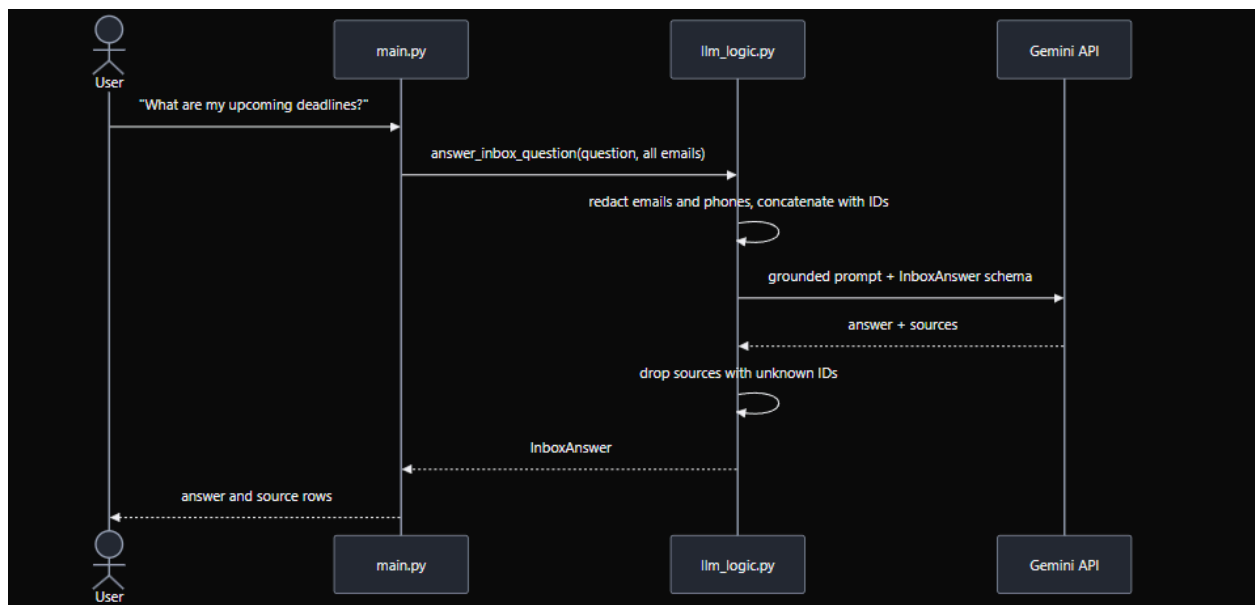
Decision	Reason
Thin UI, separate LLM module	UI and prompt work can proceed in parallel; prompts are testable without Streamlit
Pydantic schemas passed as <code>response_schema</code>	The model is constrained to a typed JSON shape; output is re-validated on receipt
Enum-valued fields (<code>Literal</code>) for priority, sentiment, tone, confidence	Drives UI colours and the health score deterministically
Two-level cache: session state, then SQLite	Avoids repeat LLM calls on reruns <i>and</i> after an app restart
SQLite repository behind one class (<code>InboxRepository</code>)	Zero-setup single-user store; the class boundary lets PostgreSQL replace it later
<code>privacy.py</code> as the single outbound boundary; every prompt function calls it	One place to strengthen redaction; tested at the LLM boundary
Consent checkbox disables every AI button	Nothing leaves the app without an explicit user action
Lazy client + single <code>_generate()</code> helper	App starts and explains a missing key; retry, logging, and error handling can live in one place
<code>.ics</code> file instead of calendar API	No OAuth or calendar credentials to protect; works with every calendar
No send capability	Enforces human-in-the-loop by construction

13.2 Request flows

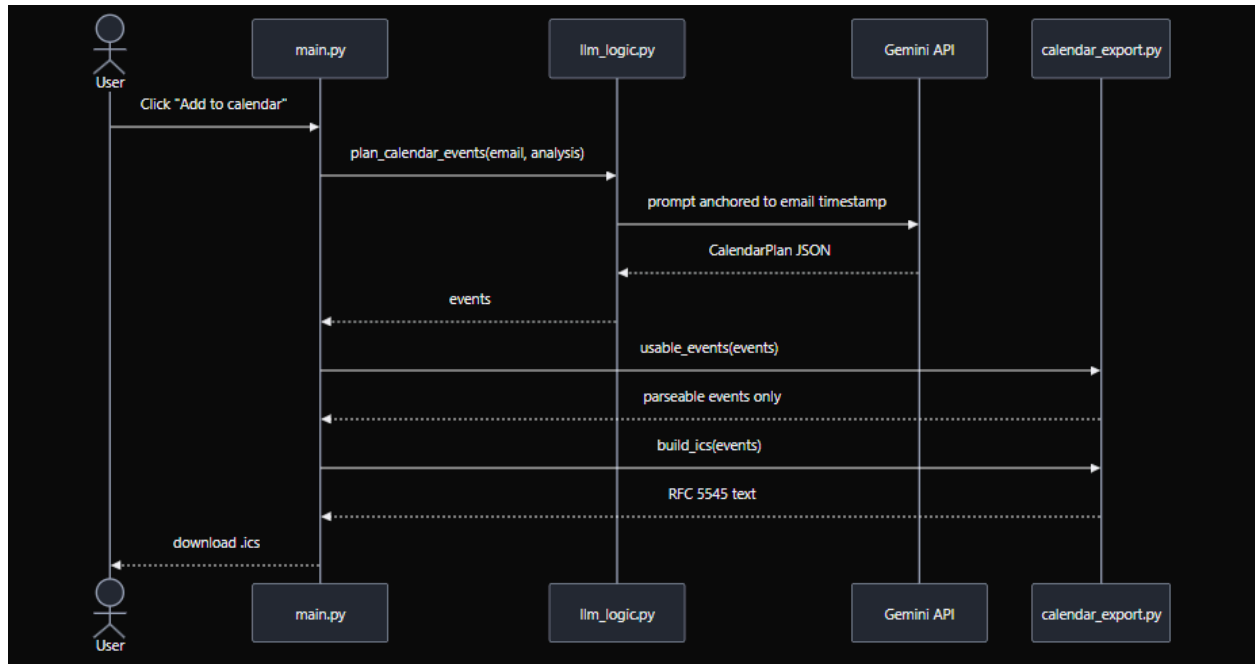
Analyse → **draft**



Ask My Inbox



Add to calendar



13.3 Component responsibilities

Component	Responsibility	Depends on
<code>main.py</code>	Page layout, consent gate, session state, restore-from-database on start, calls to LLM/storage/export layers, health-score formula	all others
<code>ui.py</code>	CSS and pure HTML-builder functions; colour maps; escapes every dynamic value passed to it	stdlib
<code>llm_logic.py</code>	Lazy client, <code>_generate()</code> (call, parse, validate, translate errors), four prompt functions, redaction before each call, citation-ID filter	<code>schemas</code> , <code>privacy</code> , Gemini SDK
<code>privacy.py</code>	<code>redact_text</code> (email and phone patterns) and	<code>schemas</code>

Component	Responsibility	Depends on
	<code>redact_email</code> (copy with masked sender address and body)	
<code>storage.py</code>	<code>InboxRepository</code> : create schema, seed emails once, save/load/clear generated artefacts as JSON	<code>schemas</code> , <code>sqlite3</code>
<code>schemas.py</code>	<code>Email</code> , <code>Task</code> , <code>EmailAnalysis</code> , <code>DraftReply</code> , <code>CalendarEvent</code> , <code>CalendarPlan</code> , <code>InboxSource</code> , <code>InboxAnswer</code>	<code>pydantic</code>
<code>calendar_export.py</code>	Parse model datetimes defensively, drop unusable events, serialise to <code>.ics</code> (CRLF, escaping, 75-octet folding)	<code>schemas</code>
<code>tests/</code>	Offline tests for calendar export, redaction, storage, and the LLM boundary (with <code>_generate</code> stubbed)	<code>pytest</code>

14. Data design

14.1 Current model (Pydantic contracts + SQLite)

Entity	Fields	Notes
<code>Email</code>	<code>id</code> , <code>sender_name</code> , <code>sender_email</code> , <code>subject</code> , <code>body</code> , <code>timestamp</code> (ISO string), <code>thread_id</code> ?	Loaded from JSON, seeded into SQLite

Entity	Fields	Notes
EmailAnalysis	summary, priority, priority_reason, tasks[], sentiment, suggested_tone	Cached per email (session + SQLite)
Task	description, deadline? (free text)	Embedded in analysis; not linked to email id
DraftReply	reply_text, confidence, confidence_reason, needs_human_review	Cached per email; user edits to the text are not saved
CalendarEvent / CalendarPlan	title, start, end, all_day, source	Times are ISO strings, timezone-less ("floating")
InboxAnswer / InboxSource	answer, sources[email_id, subject, sender]	Not persisted

Physical schema (data/inbox.db, git-ignored)

Table	Columns	Notes
emails	id PK, payload (JSON of Email)	Seeded with INSERT OR IGNORE — later edits to emails.json never overwrite an existing row (G-17)
artefacts	email_id FK → emails.id (ON DELETE CASCADE), kind ∈ {analysis, draft, calendar}, payload (JSON); PK (email_id, kind)	One row per email and kind; regenerating overwrites via upsert

Design trade-offs. Storing each Pydantic model as a JSON payload keeps schema changes cheap and reuses the same validation on read (`model_validate_json`). The cost: nothing is queryable in SQL (for example "all Urgent emails" or "tasks due this week"), there is no history, and rows carry no model or prompt version, so results go stale when prompts change. The file is unencrypted, and the original (unredacted) email bodies are stored locally; redaction applies only to what is sent to Gemini.

Database choice — PostgreSQL (with **pgvector** when retrieval is added): the domain is relational (user → mailbox → thread → email → analysis → task); LLM outputs fit typed columns with **JSONB** for anything schema-flexible; row-level security supports per-user isolation; one engine can serve both relational queries and embedding search. *Prototype store (implemented)*: SQLite via **InboxRepository** — see §14.1; the schema above is the normalised target.

Retrieval strategy (target): store embeddings per email; answer questions from the top-k relevant emails rather than the entire inbox; keep **email_id** as the citation key and look sources up in the database rather than trusting model-supplied metadata.

Improvements over the current model: **deadline_at** as a real datetime beside the original phrase; **model_version** / **prompt_version** on every AI output for auditability; **user_final_text** and **decision** capture human edits, which become a feedback signal for quality measurement.

15. LLM design

15.1 Prompt catalogue

Function	Input	Output schema	Key guardrails
analyze_email	Sender, subject, body, optional thread context	EmailAnalysis	Rubric for priority (Urgent = action today; High = this week); sentiment describes the <i>sender</i> ; tone describes the <i>reply</i>
draft_reply	Email, analysis (priority, tone, open tasks), optional thread context	DraftReply	"Do NOT invent facts, prices, or commitments"; explicit High/Medium/Low rubric; review required unless High
plan_calendar_events	Email, extracted tasks, email timestamp as "now"	CalendarPlan	"Do NOT invent dates"; empty list is valid; deadline = 30-min block ending

Function	Input	Output schema	Key guardrails
			at deadline; ISO 8601 local times
<code>answer_inbox_question</code>	Question, all emails with IDs	<code>InboxAnswer</code>	"Use ONLY the context"; fixed refusal sentence; sources must support the answer

15.2 Techniques in use

- **Schema-constrained output** — `response_mime_type="application/json"` plus a Pydantic `response_schema`, then `model_validate` on the result.
- **Task decomposition / chaining** — analysis feeds drafting; extracted tasks feed calendar planning. Each step is small, inspectable, and independently cacheable.
- **Rubric-in-prompt** — explicit definitions for priority and confidence levels reduce inconsistency.
- **Temporal grounding** — the email's own timestamp is supplied as "now", so "Monday" resolves relative to when the email arrived, not when the demo runs.
- **Abstention by design** — refusal sentence for unanswerable questions; empty calendar list when nothing is datable.
- **Model fit** — a low-cost "lite" model is adequate for extraction and classification, keeping latency and cost low.
- **Privacy boundary** — every prompt function redacts emails and phone numbers first, including the user's own question; verified by an offline test that inspects the outgoing prompt.
- **Untrusted-input instruction** — all four prompts tell the model to treat email text as data and never follow instructions inside it (a first layer; see §15.3).
- **Citation filtering** — source IDs the model returns that are not in the inbox are discarded in code.

15.3 Known limitations and mitigations

Limitation	Status	Mitigation
<code>needs_human_review</code> is decided by the model, though the rule is deterministic ("true unless High")	Open	Derive in code: <code>needs_review = confidence != "High"</code>

Limitation	Status	Mitigation
Model supplies source subject/sender	Partly fixed — unknown IDs are dropped	Return IDs only; look up subject/sender from the inbox
Untrusted email text is interpolated straight into the prompt	Partly mitigated — "treat as untrusted" line added to all prompts	Add delimiters, move instructions to <code>system_instruction</code> , validate outputs
Whole inbox sent for every question	Open	Top-k retrieval; context caching
No few-shot examples; no evaluation	Open	2–3 examples per prompt; labelled golden set (§19)
Reply has no sender identity/signature	Open	Add user profile (name, role, signature) to the draft prompt
Malformed or empty model response	Partly fixed — invalid JSON becomes a clear error	Check finish reason; retry once; also catch SDK/network errors
Thread context never supplied	Open — README now acknowledges it; <code>e1.thread_id</code> is still null	Fix the data, build context from same-thread emails, reset the local DB (G-17)
Redaction side effects	New — the phone pattern masks ISO dates such as <code>2026-10-05</code>	Tighten the pattern; add a regression test (G-13)

16. Technology stack and justification

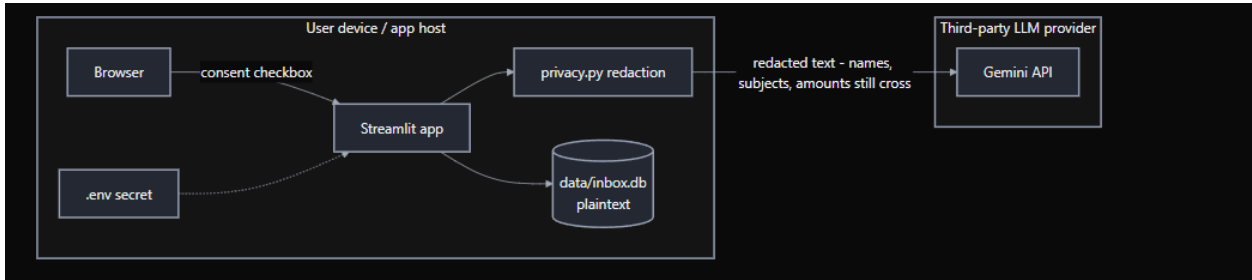
Layer	Choice	Why	Alternatives considered	Production path
UI	Streamlit	Fastest route to an interactive data/LLM demo in pure Python; built-in state and caching	React + FastAPI (more effort, more control)	React front end on a FastAPI backend

Layer	Choice	Why	Alternatives considered	Production path
LLM	Gemini <code>gemini-3.5-flash-lite</code>	Low cost and latency for classification/extraction; native structured JSON output; large context window; free tier for development	Larger Gemini tier for harder drafting; other providers	Model routing: lite for triage, stronger model for drafts; paid/enterprise tier for privacy terms
Contracts	Pydantic	Same schema drives the model output constraint and validation; typed, self-documenting	dataclasses; hand-written JSON Schema	Unchanged
Calendar	<code>.ics</code> via stdlib	No credentials or API scopes; universal import	Google Calendar API / Microsoft Graph (need OAuth + consent screen)	Optional OAuth write-back as an opt-in
Config	<code>python-dotenv</code>	Standard local secrets handling	Environment variables only	Secret manager (cloud KMS / vault)
Data	JSON seed + SQLite (<code>storage.py</code>) + session state	Zero setup, no server or credentials, results survive restarts	PostgreSQL directly; JSON only	PostgreSQL (+ <code>pgvector</code>) behind the same repository interface
Testing	<code>pytest</code> , offline (no model calls)	Fast, free, deterministic; LLM stubbed at <code>_generate</code>	Live-model tests (slow, costly, flaky)	Add a separate, opt-in LLM evaluation suite

Layer	Choice	Why	Alternatives considered	Production path
Dependencies	5 packages (adds <code>pytest</code>)	Small attack surface, fast install	—	Pin versions in <code>requirements.txt</code> (currently unpinned)

17. Security, privacy, and PII

17.1 Trust boundaries



The sensitive crossing is still **email content** → **Gemini API**. It is now gated by consent and passes through redaction, but only email addresses and phone numbers are masked; names, subjects, amounts, and business details are sent (the README says so plainly).

17.2 Threat model

ID	Threat	Status now	Required control
T1	Secret leakage	Mostly closed — <code>.env.example</code> is a placeholder, <code>.env</code> is ignored, no key in any commit	Rotate the key that was in the archive shared earlier; add pre-commit secret scanning; never zip <code>.env</code>
T2	Prompt injection — an email says "ignore previous instructions..."	Partly mitigated — untrusted-data instruction in all prompts	Delimit and label untrusted text; <code>system_instruction</code> ; output validation; keep the model tool-less; human review before any send

ID	Threat	Status now	Required control
T3	Markup injection / XSS — LLM text or a subject rendered as raw HTML	Still vulnerable — <code>main.py</code> L187 (<code>result.answer</code>) and L326 (<code>selected_email.subject</code>); every builder in <code>ui.py</code> escapes correctly	Wrap both in <code>html.escape(...)</code> — a two-line fix
T4	PII disclosure to a third party	Partly mitigated — consent gate; emails and phones masked, including in the user's question	Mask more (names, subjects if required); fix the date bug (G-13); paid/enterprise tier whose terms exclude training (<i>verify the provider's current terms; free-tier terms differ</i>); DPA and data residency
T5	Hallucinated or wrong content acted on	Partly mitigated — confidence flag, "do not invent", citation IDs validated	Derive the review flag in code; keep the no-auto-send rule
T6	Unauthorised access	Unchanged — no authentication	SSO/OAuth; per-user isolation
T7	Data retention	Changed — data is now persisted in plaintext <code>data/inbox.db</code> (git-ignored): original email bodies plus generated summaries and drafts; no encryption, expiry, or "delete my data" control; consent is not recorded	Add a clear-local-data control, expiry, encryption at rest for real data; store consent with a timestamp

ID	Threat	Status now	Required control
T8	Over-broad mailbox permissions	None requested	Read-only scopes first when a connector is added

17.3 Privacy principles

- **Consent first** — no model request until the user ticks the consent box (implemented).
- **Minimisation** — send the least text needed; today only contact identifiers are masked, which is narrower than this principle requires.
- **Human in the loop** — no automated outbound action.
- **Transparency** — show *why* for every AI judgement (priority reason, confidence reason).
- **User control** — drafts are editable; calendar export is an explicit download.
- **Honest claims** — the README states this is a demonstrator, not a compliance claim; keep that wording in the pitch.
- **Compliance** — if real user data is processed, assess obligations under applicable law (for example India's Digital Personal Data Protection Act 2023, or GDPR for EU data subjects) with legal counsel.

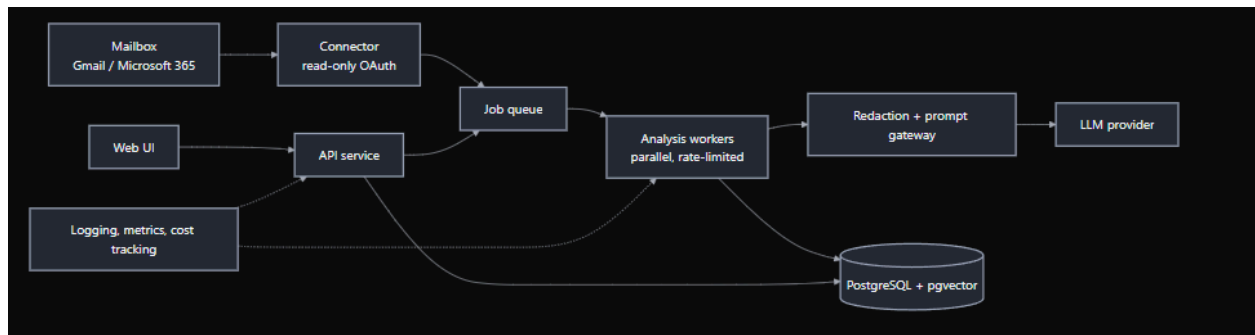
18. Reliability, scalability, and performance

18.1 Current behaviour

- Each action makes one LLM call; results are cached in session state and persisted in SQLite, so a restart does not re-bill the API.
- "Analyze all" runs calls **sequentially**, with **no timeouts or retries**.
- Error handling is **partial**: invalid JSON and a missing key produce a clear `RuntimeError`, which only the Ask My Inbox handler catches. Analyse, draft, calendar, and "Analyze all" have no handler, and SDK or network errors (for example a rate-limit response) are not converted to `RuntimeError`, so they can still surface as a raw Streamlit exception (G-14).
- `calendar_export` is defensive: unparseable datetimes are skipped, end $\leq$ start is clamped, timezone offsets are stripped, `.ics` output uses CRLF, escaping, and line folding.
- On startup the app loads saved results with one query per email and opens a new SQLite connection each time — fine at 8 emails, wasteful at scale (G-19).

18.2 Production target

flowchart LR



Concern	Approach
Throughput	Queue + parallel workers; use the provider's batch mode for backlog analysis
Rate limits	Token-bucket limiter; exponential backoff with jitter
Cost	Cache by content hash; lite model for triage; retrieval instead of full-inbox context
Latency	Analyse on arrival (background) so the UI reads stored results
Resilience	Retry once on malformed JSON; fall back to "analysis unavailable" state
Observability	Log latency, token counts, model/prompt versions, and failures — never log raw email bodies

19. Testing and quality strategy

ID	Test	Type	Pass condition	Status
T-UNIT-01	Health score with 0, 1, many stressors; floor at 0	Unit	Matches formula; never < 0	Not yet
T-UNIT-02	<code>usable_events</code> drops unparseable starts	Unit	Invalid event removed, valid kept	Automated (<code>test_calendar_export.py</code>). Clamp of end

ID	Test	Type	Pass condition	Status
				≤ start checked manually only
T-UNIT-03	<code>_escape</code> handles <code>,</code> <code>;</code> <code>\</code> newline	Unit	Output is RFC 5545-safe	Automated (same file: summary and description escaping, CRLF)
T-UNIT-04	<code>_fold</code> never splits a UTF-8 character; lines ≤ 75 octets	Unit	Holds for multi-byte text	Not yet
T-UI-01	Every <code>ui.py</code> builder escapes <code><script></code> in every argument	Unit	No raw tag in output	Not yet
T-PRIV-01	<code>redact_text</code> / <code>redact_email</code> mask email and phone, keep subject	Unit	Exact expected output	Automated (<code>test_privacy.py</code>)
T-PRIV-02	Dates survive redaction (<code>2026-10-05</code> , <code>2026-09-18 16:45</code>)	Unit	Text unchanged	Not yet — currently fails (G-13)
T-PRIV-03	Outgoing inbox prompt contains no address or phone, including from the question	Boundary	Prompt clean	Automated (<code>test_llm_boundary.py</code>)
T-STORE-01	Seed once, persist, reload, clear an analysis	Integration	Round-trips equal; seed never overwrites	Automated (<code>test_storage.py</code>)

ID	Test	Type	Pass condition	Status
T-LLM-01	Analyse e1...e8 against expected priority/sentiment	Eval (golden set)	≥ 85% agreement	Not yet
T-LLM-02	Drafts for e2 and e4	Eval	Review flag true; no invented price/date	Not yet
T-LLM-03	Out-of-scope inbox question	Eval	Exact refusal sentence; no sources	Not yet
T-LLM-04	Returned source IDs exist in the inbox	Boundary	Unknown IDs removed	Automated (<code>test_llm_boundary.py</code> , model stubbed)
T-LLM-05	Calendar for e5, e6	Eval	e5 → 0 events; e6 → Mon 21 Sep 15:00	Not yet
T-LLM-06	Thread context for e8	Eval	Summary mentions the outage	Not yet
T-SEC-01	Injection email in the dataset	Adversarial	Analysis/draft ignore embedded instructions	Not yet
T-SEC-02	Subject containing <code></code>	Adversarial	Rendered as text	Not yet — would fail at <code>main.py</code> L326
T-SEC-03	Repository secret scan	CI	0 findings	Manual pass (no key in any commit); not automated

ID	Test	Type	Pass condition	Status
T-SEC-04	Search codebase for any send/SMTP capability	Review	None present	Manual pass
T-REL-01	Simulated API failure / timeout / malformed JSON	Fault injection	Friendly message, app remains usable	Not yet

Result today: 5 offline tests pass; 8 of the 20 planned checks have automation or a manual pass.

Test data: extend `data/emails.json` (and remember to reset `data/inbox.db`, G-17) with edge cases — very short email, non-English email, sarcasm, injection attempt, email with attachment reference, long thread, no-date deadline ("ASAP").

20. Deployment view

- **Prototype:** `streamlit run main.py` on a laptop; `GEMINI_API_KEY` from a local, untracked `.env`; local `data/inbox.db`; tests with `python -m pytest -q`.
- **Pilot:** container on a managed service behind SSO; key in a secret manager; managed database; outbound access limited to the LLM endpoint.
- **Production:** separate web, API, and worker services; managed PostgreSQL; centralised logging; CI running the tests and a secret scan on every commit.

21. Roadmap

Phase	Deliverables	Progress
0 — Before submission (hours)	Rotate the API key · fix the 2 unescaped renders · error handling on every LLM call · fix <code>e1.thread_id</code> and wire thread context · derive review flag in code · validate sources	Key removed from repo, but rotation is yours to do · unescaped renders open · error handling partial · thread work open · review flag open · source IDs done , metadata open
1 — Hardening (days)	Unit tests · golden-set eval · delimiters and <code>system_instruction</code> ·	Tests started (5) · eval open · delimiters partial (instruction line only) ·

Phase	Deliverables	Progress
	SQLite persistence · normalise health score	persistence done · health score open
2 — Pilot (weeks)	Read-only Gmail / Microsoft 365 connector · PostgreSQL · broader redaction · SSO · audit log	Not started
3 — Scale (months)	Queue + workers · retrieval with embeddings · feedback loop from user edits · multi-language	Not started

Highest-value next five, in order: (1) `html.escape` the two renders; (2) fix the ISO-date redaction bug and add its test; (3) wrap `get_analysis`, `get_draft`, `get_calendar` and SDK errors in one handler; (4) fix `e1.thread_id`, pass thread context, and reset `data/inbox.db`; (5) derive `needs_human_review` in code.

22. Known gaps in the current build

Status is measured against repository commit `f16119f`.

ID	Gap	Status	Fix
G-01	API key in <code>.env.example</code> ; <code>.env</code> shipped	Closed in repo — rotate the previously shared key	Placeholder only; secret scanning
G-02	Unescaped output at <code>main.py</code> L187 and L326	Open	<code>html.escape</code>
G-03	No error handling around LLM calls	Partial — Ask My Inbox only (see G-14)	Central handler
G-04	<code>thread_context</code> never passed; <code>e1</code> has no <code>thread_id</code> although its follow-up <code>e8</code> does	Open (README now says so)	Data fix + build context in <code>main.py</code>

ID	Gap	Status	Fix
G-05	<code>needs_human_review</code> is model-decided	Open	Derive from <code>confidence</code>
G-06	Source metadata trusted from the model	Partial — unknown IDs dropped	Look up subject/sender by <code>email_id</code>
G-07	Bare <code>KeyError</code> when the key is missing	Closed — lazy client, clear message	—
G-08	Health score not normalised to inbox size: 5 urgent emails zero an 8-email inbox, and a large inbox always scores 0	Open	Use ratios (share of urgent/high)
G-09	Four near-identical <code>generate_content</code> blocks	Closed — <code>_generate()</code>	—
G-10	No tests; dependencies unpinned	Partial — 5 tests; versions (including <code>pytest</code>) still unpinned	Add health-score, <code>ui.py</code> , <code>_fold</code> tests; pin versions
G-11	Attachments referenced (e2) but not processed	Open	State as a limitation or add document parsing
G-12	README in starter-template voice	Closed — rewritten	—
G-13	New. Phone-number redaction also masks ISO dates: <code>2026-10-05</code> → <code>[redacted-phone]</code> , <code>2026-09-18 16:45</code> →	Open — high (can break deadline extraction and calendar export)	Require a leading <code>+</code> or phone-style grouping; exclude <code>\d{4}-\d{2}-\d{2}</code> ; add a regression test

ID	Gap	Status	Fix
	[redacted-phone]:45. The sample emails use natural-language dates, so the tests never catch it		
G-14	New. Only Ask My Inbox has a handler (<code>RuntimeError</code>). <code>main.py</code> L116, L256, L288, L331 are unguarded, and SDK/network errors are not <code>RuntimeError</code>	Open — high	Handle inside the <code>get_*</code> helpers; catch the SDK error type; retry with backoff
G-15	New. Redaction is narrow: subjects, sender names, amounts, and company names still go to Gemini	Open (disclosed in README)	Widen, or keep the disclosure in the pitch; use a paid tier
G-16	New. Local DB is plaintext with no delete control or expiry; consent is not persisted; results carry no model/prompt version and go stale when prompts change	Open	"Clear local data" button; version column
G-17	New. Seeding uses <code>INSERT OR IGNORE</code> , so fixing <code>emails.json</code> (for example <code>e1.thread_id</code>) never reaches an	Open	Delete the DB after data changes, or upsert seed data in demo mode

ID	Gap	Status	Fix
	existing <code>data/inbox.db</code>		
G-18	New. README "Known limits" still says nothing persists; its module list omits <code>privacy.py</code> , <code>storage.py</code> , and <code>tests/</code> . <code>docs/BRD.md</code> and <code>docs/HLD.md</code> are about 40 lines and omit the threat model, data model, and test plan	Open	Update the README; replace <code>docs/</code> with this document
G-19	New, low. SQLite connections are opened per call and not explicitly closed (<code>with</code> commits but does not close); saved results are loaded one query per email on every start	Open — low	<code>contextlib.closing</code> or one connection; batch load